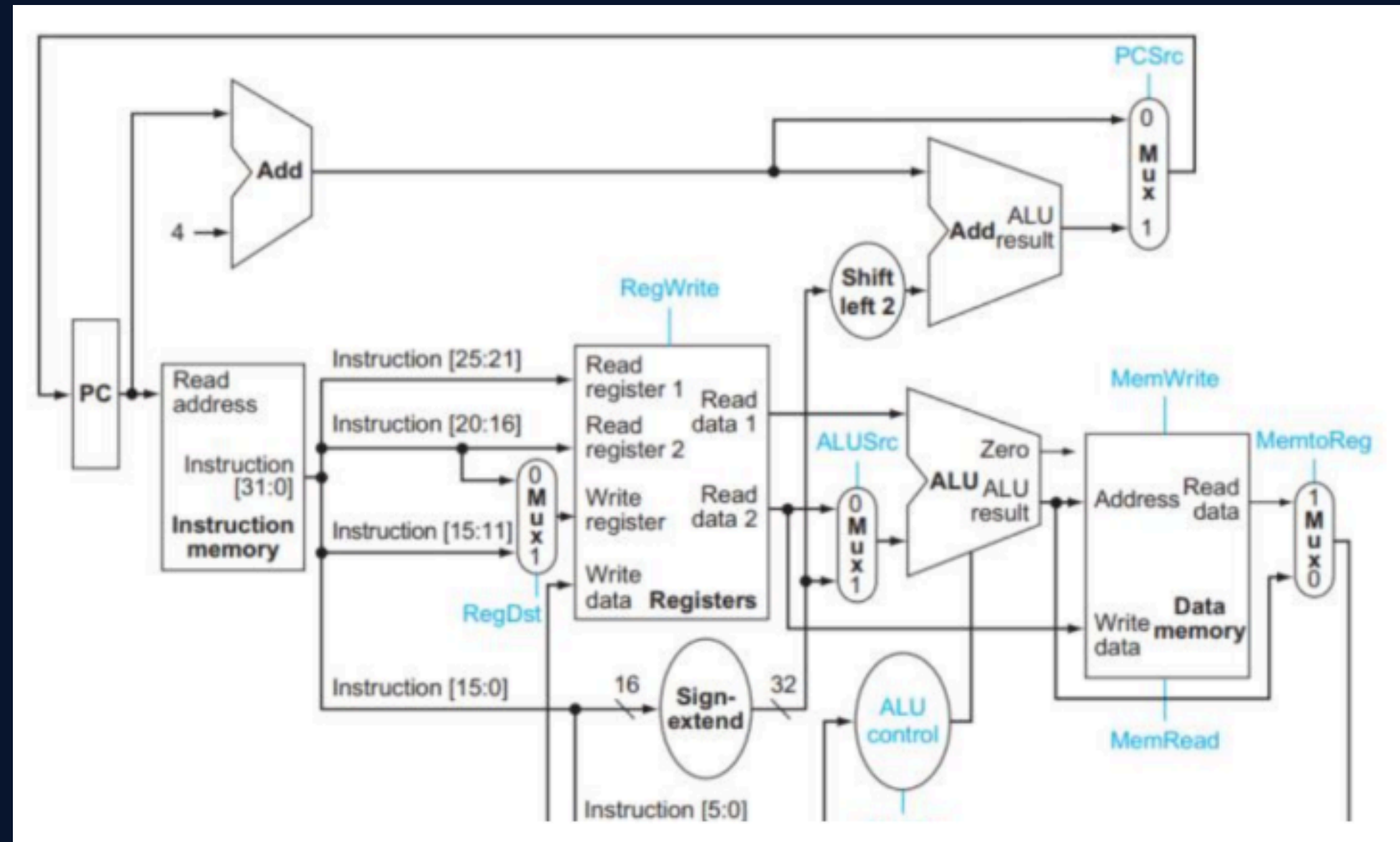


FASE 1

PROYECTO FINAL

RECAMIER ACEVEDO JOSÉ DE JESÚS
GARCÍA TORRES DANIELA CATALINA
ESPINOZA ROBLES FATIMA GABRIELA
LIZAOLA HERNÁNDEZ AYLIN DANAE
LOMELÍ MEJÍA MARÍA DEL SAGRARIO
FERRER SAUCEDO VALERIA LIZETH

El proyecto busca realizar una parte de un procesador MPS de 32 bits (primera parte del proyecto final), con las siguientes características



Pero antes que nada, empecemos con algunas definiciones clave del procesador MIPS de 32 bits.



SET DE INSTRUCCIONES (ISA)

Una ISA (por sus siglas en inglés, Instruction Set Architecture o Arquitectura del Conjunto de Instrucciones) es el conjunto de instrucciones básicas, registros y reglas que un procesador entiende y puede ejecutar. Funciona como un "puente" o traductor fundamental entre el hardware físico del procesador y el software. Define lo siguiente:

- Instrucciones: Las operaciones básicas que puede realizar la CPU (sumar, restar, mover datos, operaciones lógicas).
- Registros: Las pequeñas áreas de almacenamiento temporal dentro del procesador.
- Formato de los datos: Cómo se codifican los datos (por ejemplo, en 32 o 64 bits).

Tipos de arquitectura

- **RISC:** Una computadora con conjunto de instrucciones reducido (RISC) es un tipo de arquitectura de microprocesador que adopta simplicidad y velocidad.
- **CISC:** Computadora de conjunto de instrucciones complejas (CISC) es una arquitectura de microprocesador que intenta mejorar el rendimiento mediante el uso de instrucciones complejas.

CARACTERÍSTICAS GENERALES DE MIPS DE 32 BITS

MIPS (Microprocessor without Interlocked Pipeline Stages) es una arquitectura de tipo RISC (Reduced Instruction Set Computer) con las siguientes propiedades:

01

Longitud fija: Todas las instrucciones miden exactamente 32 bits (4 bytes).

02

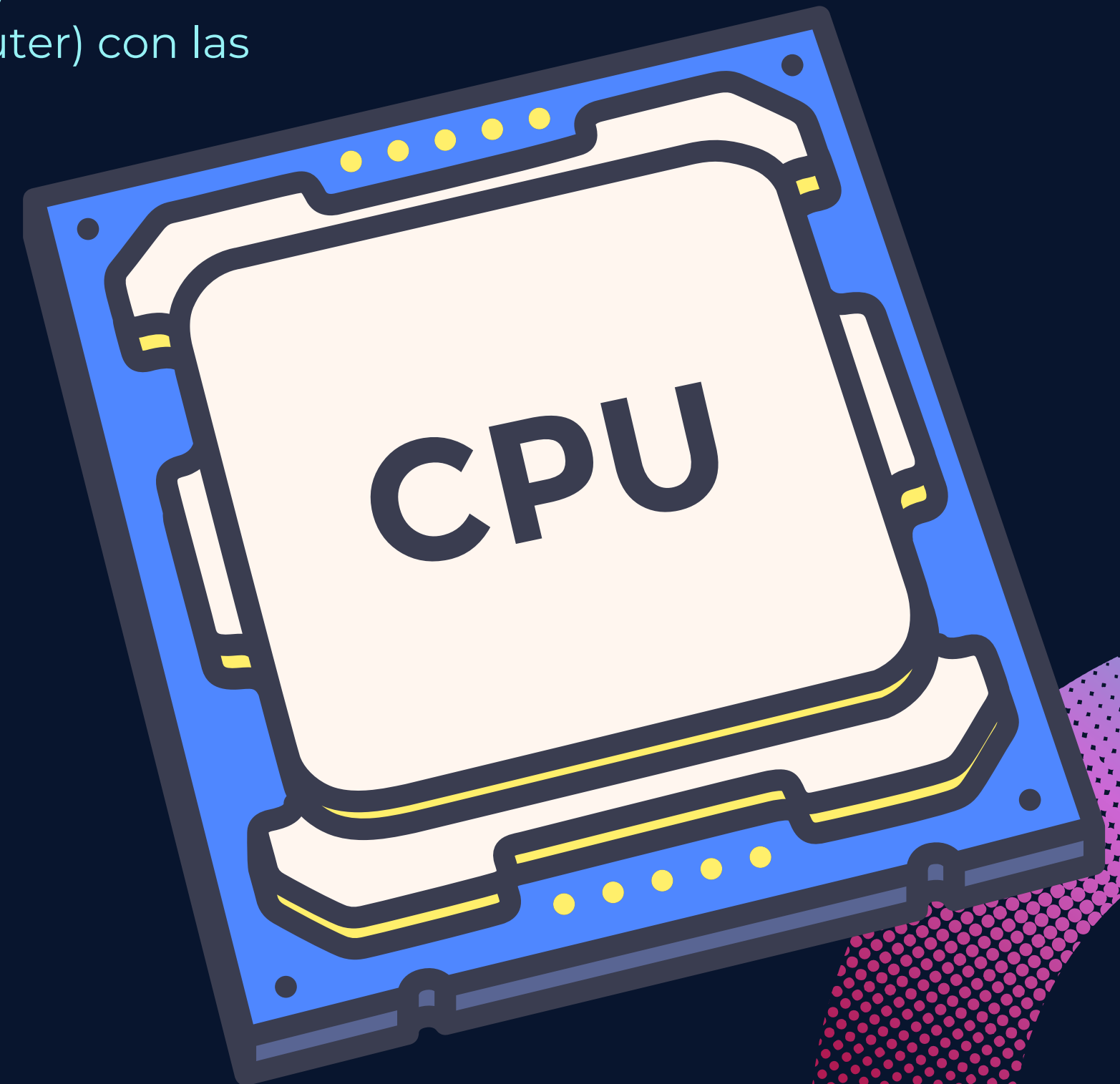
Arquitectura Load-Store: Solo las instrucciones de carga y almacenamiento acceden a la memoria ram; las operaciones aritméticas operan estrictamente entre registros.

03

Direccionamiento de memoria: Utiliza direccionamiento por bytes y estructuración nativa Big-Endian o Little-Endian.

04

Segmentación (Pipelining): Diseñado nativamente para ejecutar instrucciones en un ciclo de reloj mediante un pipeline clásico de 5 etapas (IF, ID, EX, MEM, WB).



ELEMENTOS PRINCIPALES DEL PROCESADOR

01

Banco de Registros

Contiene 32 registros de propósito general (del \$0 al \$31), cada uno de 32 bits. El registro \$0 (\$zero) está cableado permanentemente al valor cero.

02

Contador de Programa (PC)

Registro dedicado que almacena la dirección de memoria de la próxima instrucción a ejecutar.

03

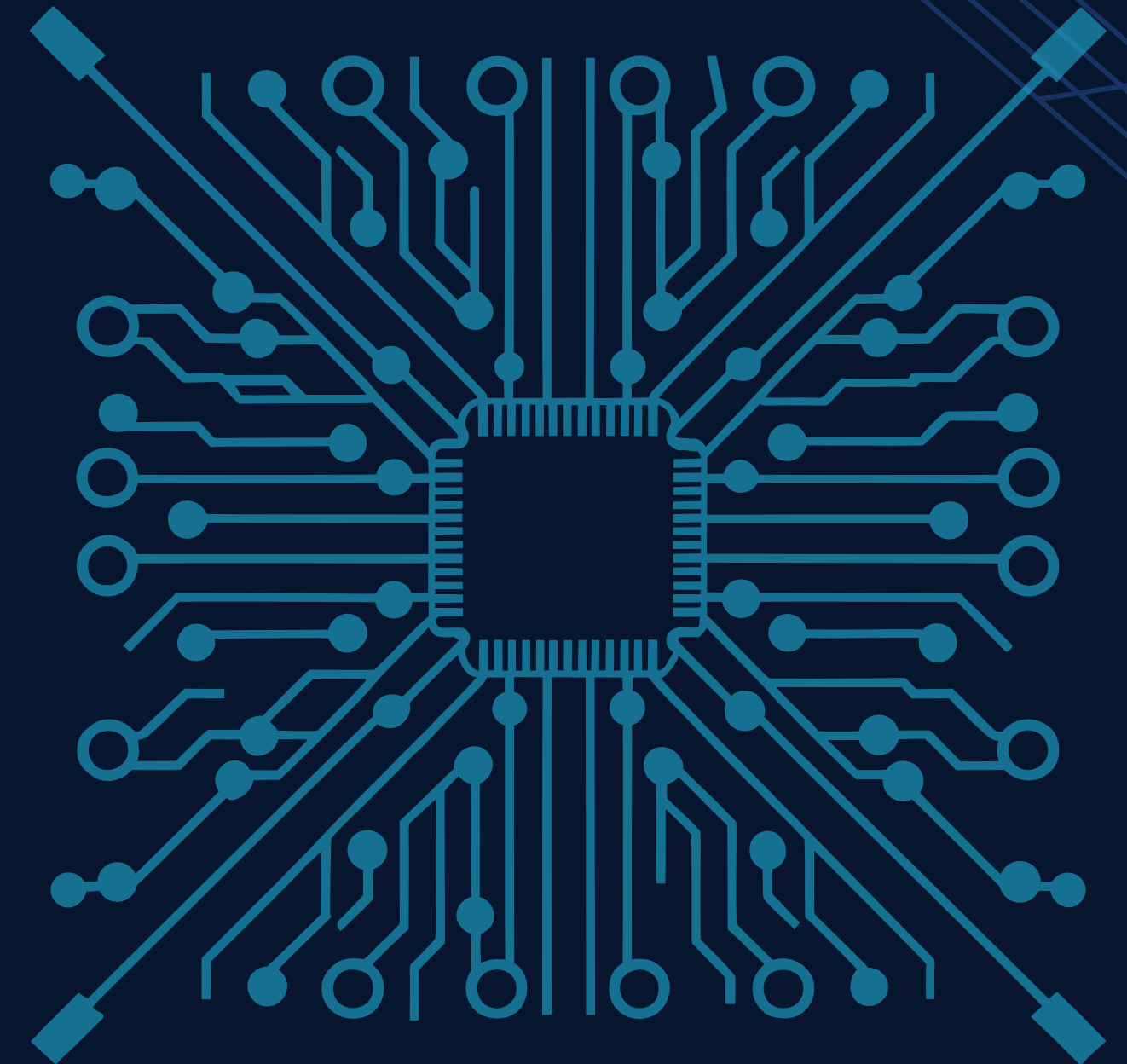
Unidad Aritmético Lógica (ALU)

Bloque funcional que realiza las operaciones matemáticas (suma, resta) y lógicas (AND, OR, XOR).

04

Unidad de Control

Decodifica las instrucciones y genera las señales eléctricas necesarias para coordinar los componentes del procesador.



FORMATOS DE INSTRUCCIÓN (CODIFICACIÓN DE 32 BITS)

Las instrucciones MIPS se dividen estrictamente en tres formatos según la distribución de sus bits:

TIPO R (REGISTRO)

Utilizado para operaciones aritméticas y lógicas puras entre registros.

- opcode (6 bits): Código de operación (siempre es 000000 para Tipo R).
- rs (5 bits): Primer registro fuente.
- rt (5 bits): Segundo registro fuente.
- rd (5 bits): Registro destino (donde se guarda el resultado).
- shamt (5 bits): Cantidad de desplazamiento (shift amount), usado en operaciones de bits.
- funct (6 bits): Especifica la función exacta de la operación (ej. suma, resta).



FORMATOS DE INSTRUCCIÓN (CODIFICACIÓN DE 32 BITS)

Tipo I (Inmediato)

Utilizado para transferencias de datos, saltos condicionales y operaciones con constantes.

- opcode (6 bits): Código de la operación específica.
- rs (5 bits): Registro fuente o registro base de memoria.
- rt (5 bits): Registro destino o registro de datos a transferir.
- immediate (16 bits): Valor constante o desplazamiento de memoria (-32,768 a 32,767).

Tipo J (Salto / Jump)

Utilizado para saltos incondicionales de largo alcance.

- opcode (6 bits): Código de operación de salto.
- address (26 bits): Dirección objetivo del salto en la memoria del programa.



SET DE INSTRUCCIONES ESPECÍFICO DEL PROYECTO



ARITMÉTICAS Y
LÓGICAS (TIPO R)

ADD, SUB,
MUL, DIV, OR,
AND, SLT,
NOP.

ACCESO A MEMORIA
(TIPO I)

LW, SW.

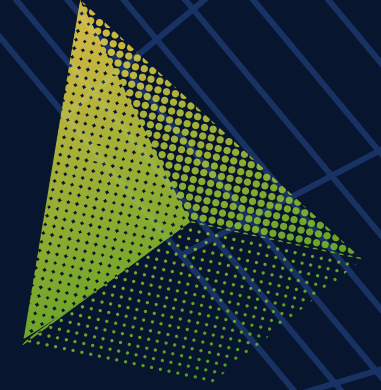
ARITMÉTICAS Y
LÓGICAS (TIPO I)

ADDI, SUBI,
ORI, ANDI,
SLTI.

CONTROL DE
FLUJO (TIPO I Y J)

BEQ, BNE,
BGTZ (TIPO I)
Y J (TIPO J).

DEFINICIONES DE SET DE INSTRUCCIONES



- add: Suma (Tipo R).
 - sub: Resta (Tipo R).
 - mul: Multiplicación (Tipo R)
 - and / or / xor: Operaciones lógicas bit a bit (Tipo R).
 - div: División (Tipo R)
 - sll / srl: Desplazamientos lógicos a la izquierda o derecha (Tipo R).
 - nop: No operation (Tipo R)
-
- addi: Suma un valor inmediato constante (Tipo I).
 - ori: Operación lógica con un valor inmediato (Tipo I).
 - slt / slti: Establece un registro en 1 si es menor que otro valor, si no en 0 (Tipo R / Tipo I).
 - lw (Load Word): Copia una palabra de 32 bits desde la memoria hacia un registro.(Tipo I).
 - sw (Store Word): Copia una palabra de 32 bits desde un registro hacia la memoria.(Tipo I).
 - beq (Branch if Equal): Salta a una dirección relativa si dos registros son iguales (Tipo I).
 - bne (Branch if Not Equal): Salta a una dirección relativa si dos registros son diferentes (Tipo I).
 - bgtz (Branch on Greater Than Zero): Saltar si es Mayor que Cero (Tipo I).
-
- j (Jump): Salta directamente a una dirección absoluta de 26 bits (Tipo J).

DESARROLLO DE LOS ELEMENTOS IMPLEMENTADOS (FASE 1)

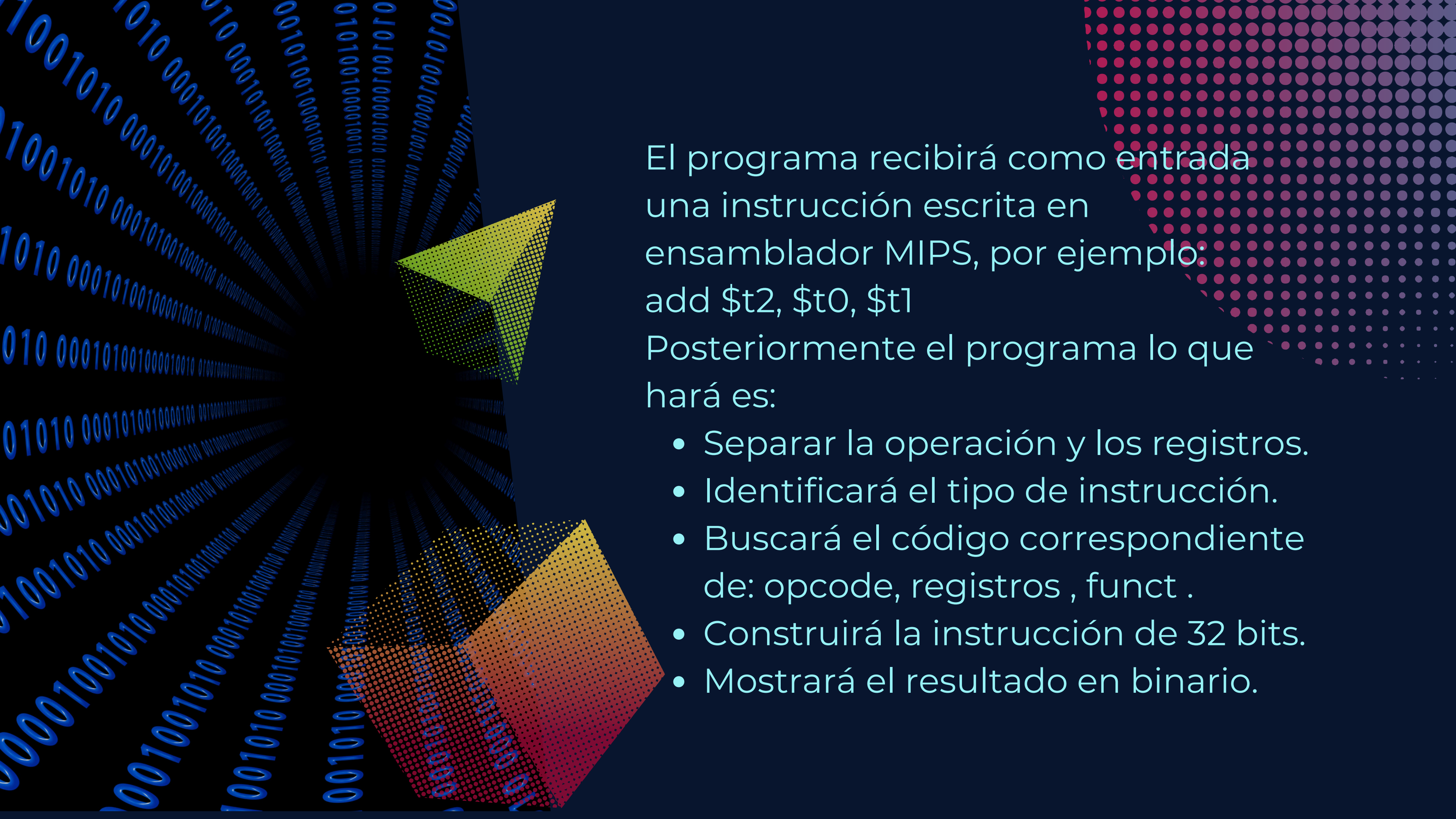
En esta primera fase del proyecto, el objetivo principal fue la implementación del single datapath (ruta de datos de un solo ciclo) de MIPS de 32 bits, enfocado exclusivamente en la ejecución de instrucciones de Tipo R.

Para lograr esto, se desarrollaron e interconectaron los siguientes módulos de hardware (basados en la Figura 1 y 2 del documento) :

- **Program Counter (PC):** Registro que almacena la dirección de la instrucción actual.
- **Memoria de Instrucciones:** Módulo de solo lectura donde se precargan las instrucciones en código máquina.
- **Banco de Registros (Registers):** Memoria interna que contiene los 32 registros. Para las instrucciones Tipo R, lee dos registros fuentes (Read register 1 y Read register 2) y escribe en un registro destino (Write register).
- **ALU (Arithmetic Logic Unit):** Encargada de realizar las operaciones matemáticas (suma, resta) y lógicas (AND, OR) requeridas por la instrucción.
- **Unidades de Control (Control y ALU Control):** La unidad de control principal decodifica los 6 bits más significativos (Opcode). Para las instrucciones Tipo R, delega la decisión final al ALU Control, el cual lee los últimos 6 bits de la instrucción (campo funct) para determinar la operación exacta que la ALU debe realizar.

PROPUESTA DE ALGORITMO

Como parte del proyecto final, se propone desarrollar un programa en Python capaz de traducir instrucciones en ensamblador MIPS tipo R hacía código máquina binario. El objetivo de este programa es generar instrucciones binarias que posteriormente serán cargadas en la memoria de instrucciones del datapath implementado en Verilog, permitiendo verificar el correcto funcionamiento de cada parte del programa.

The background features a dark blue field with diagonal lines of glowing blue binary code (0s and 1s) on the left side. On the right, there are two overlapping geometric shapes: a yellow triangle pointing upwards and a larger orange-to-red gradient triangle pointing downwards, both with a halftone dot pattern.

El programa recibirá como entrada una instrucción escrita en ensamblador MIPS, por ejemplo:

`add $t2, $t0, $t1`

Posteriormente el programa lo que hará es:

- Separar la operación y los registros.
- Identificará el tipo de instrucción.
- Buscará el código correspondiente de: opcode, registros , funct .
- Construirá la instrucción de 32 bits.
- Mostrará el resultado en binario.

Las instrucciones tipo R en MIPS utilizan el siguiente formato

opcode	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Todas las instrucciones utilizaran opcode=000000

Campo	Valor
opcode	000000
rs (\$t0)	01000
rt (\$t1)	01001
rd(\$t2)	01010
shamt	00000
funct (add)	100000

Y el código en máquina que se generaría es:
00000001000010010101000000100000 Instrucciones tipo R

Instrucción	Funct (hex)	Operación	Sintaxis
add	0x20	$\$rd = \$rs + \$rt$	add \$rd, \$rs, \$rt
sub	0x22	$\$rd = \$rs - \$rt$	sub \$rd, \$rs, \$rt
and	0x24	$\$rd = \$rs \& \$rt$	and \$rd, \$rs, \$rt
or	0x25	$\$rd = \$rs \$rt$	or \$rd, \$rs, \$rt
slt	0x2A	$rd = (rs < \$rt)$	slt \$rd, \$rs, \$rt
nop	0x00	No opera (sll \$0,\$0,0)	nop
nor	0x27	$rd = (rs \ \ \$rt)$	nor \$rd, \$rs, \$rt
xor	0x26	$\$rd = \$rs \wedge \$rt$	xor \$rd, \$rs, \$rt
sll	0x00	$\$rd = \$rt \ll \text{shamt}$	sll \$rd, \$rt, shamt
srl	0x02	$\$rd = \$rt \gg \text{shamt}$	srl \$rd, \$rt, shamt
sltu	0x2B	$rd = (rs < \$rt)$ sin signo	sltu \$rd, \$rs, \$rt



BIBLIOGRAFÍAS

Arm Ltd. (s. f.). What is Instruction Set Architecture (ISA)? Arm | The Architecture For The Digital World.
<https://www.arm.com/glossary/isa>

Arm Ltd. (s. f.). What is Instruction Set Architecture (ISA)? Arm | The Architecture For The Digital World.
<https://www.arm.com/glossary/isa>

MIPS architecture overview. (s. f.).
<https://tams.informatik.uni-hamburg.de/applets/hades/webdemos/mips.html>

Spiryn, J. (s. f.). What Is RISC and CISC Architecture & Their Differences.
<https://conclusive.tech/glossary/what-is-risc-and-cisc-architecture-their-differences/>